Artificial Intelligence: From Machine Learning to Generative AI and Agentic AI

1. Introduction

Artificial Intelligence, commonly known as AI, is a field of computer science focused on creating systems that can perform tasks normally associated with human intelligence. These tasks include understanding language, recognizing images, making decisions, solving problems, learning from experience, and generating new content.

AI has evolved significantly over the past several decades. Early AI systems were based primarily on manually written rules. Modern AI systems, however, are increasingly based on machine learning, deep learning, large-scale neural networks, and massive amounts of data.

Today, AI is used in many areas, including healthcare, finance, education, transportation, software development, entertainment, cybersecurity, and scientific research.

The development of AI can be understood through several major stages:

1. Traditional rule-based systems

2. Machine learning

3. Deep learning

4. Natural language processing

5. Transformers

6. Large Language Models

7. Generative AI

8. Retrieval-Augmented Generation

9. AI Agents and Agentic AI

Each stage builds on ideas from previous technologies while introducing new capabilities.

2. Traditional Artificial Intelligence

Early artificial intelligence systems were often based on explicit rules written by programmers.

For example, a simple rule-based system might use the following logic:

IF temperature is greater than 38 degrees Celsius, THEN indicate possible fever.

IF a customer has not paid for more than 30 days, THEN send a payment reminder.

These systems are predictable because the programmer explicitly defines the rules. However, they have significant limitations.

A rule-based system cannot easily handle situations that were not anticipated by its developers. As the number of possible situations increases, the number of rules can become extremely large and difficult to maintain.

This limitation led researchers to explore systems that could learn patterns automatically from data.

3. Machine Learning

Machine Learning is a branch of artificial intelligence in which computers learn patterns from data instead of relying entirely on manually programmed rules.

A traditional program can be represented as:

Input + Rules → Output

Machine learning changes this process.

Training Data + Expected Output → Machine Learning Algorithm → Model

Once the model has learned patterns from the training data, it can receive new input and generate predictions.

For example, a machine learning model can be trained using information about houses, including:

- Number of bedrooms
- Location
- Area in square feet
- Number of bathrooms
- Age of the property

The model can then predict the approximate price of a new house.

Machine learning is useful because real-world problems often contain patterns that are too complex to describe using simple rules.

4. Types of Machine Learning

Machine learning is commonly divided into several major categories.

4.1 Supervised Learning

Supervised learning uses labeled training data.

For example, an email classification system may be trained using examples such as:

Email: "Congratulations! You won a prize."

Label: Spam

Email: "Meeting scheduled for tomorrow."

Label: Not Spam

The model learns the relationship between the input and the correct output.

Common supervised learning tasks include:

- Classification
- Regression
- Spam detection
- Image classification
- Fraud detection
- Medical diagnosis

Classification predicts categories.

For example:

Transaction → Fraud or Not Fraud

Regression predicts continuous values.

For example:

House Information → Predicted Price

4.2 Unsupervised Learning

Unsupervised learning uses data without predefined labels.

The goal is often to discover hidden patterns or groups.

For example, an online shopping company may have information about thousands of customers. An unsupervised learning algorithm might automatically group customers based on purchasing behavior.

Possible groups could include:

- Frequent buyers
- Occasional buyers
- High-value customers
- Discount-focused customers

A common unsupervised learning technique is clustering.

4.3 Reinforcement Learning

Reinforcement learning involves an agent interacting with an environment.

The agent performs actions and receives rewards or penalties.

The basic process is:

Agent → Action → Environment → Reward → Agent Learns

The goal is to learn a strategy that maximizes long-term rewards.

Reinforcement learning has been used in areas such as:

- Game playing
- Robotics
- Resource optimization
- Autonomous systems

5. Deep Learning

Deep Learning is a specialized area of machine learning based on artificial neural networks with multiple layers.

The inspiration for neural networks originally came from simplified ideas about how biological neurons process information.

A basic artificial neuron receives inputs, processes them using mathematical operations, and produces an output.

A neural network typically contains:

- Input layer
- Hidden layers
- Output layer

Deep learning uses many hidden layers, allowing models to learn increasingly complex representations.

For example, in image recognition:

The first layers may detect simple edges.

Later layers may identify shapes.

Even deeper layers may recognize objects such as faces, cars, animals, or buildings.

Deep learning has become particularly successful because of:

- Increased computing power
- Large datasets
- Graphics Processing Units
- Improved algorithms
- Distributed computing

6. Natural Language Processing

Natural Language Processing, often abbreviated as NLP, is the field of AI focused on enabling computers to understand, process, and generate human language.

Human language is difficult for computers because it contains:

- Ambiguity
- Context
- Multiple meanings
- Grammar variations
- Cultural references

- Idioms
- Incomplete sentences

For example, consider the word "bank."

It could refer to:

- A financial institution
- The side of a river

The correct meaning depends on context.

NLP systems attempt to understand this context.

Common NLP applications include:

- Chatbots
- Translation
- Sentiment analysis
- Text summarization
- Question answering
- Speech recognition
- Text generation
- Information extraction

7. Tokens and Tokenization

Before a language model can process text, the text must usually be converted into smaller units called tokens.

A token is not always exactly the same as a word.

For example:

"Artificial Intelligence is powerful."

May be divided into tokens representing individual words or parts of words.

Tokenization is important because language models operate on numerical representations rather than raw text.

A tokenizer converts text into tokens.

The model then processes these tokens mathematically.

Different models may use different tokenization strategies.

Token limits are also important because a language model can process only a limited amount of information in its context window at one time.

8. Embeddings

Embeddings are numerical representations of data.

In natural language processing, embeddings convert text into vectors containing numerical values.

For example:

"Artificial Intelligence"

might be represented conceptually as:

[0.12, -0.45, 0.78, 0.21, ...]

The actual vectors usually contain hundreds or thousands of dimensions.

The important idea is that embeddings attempt to capture semantic meaning.

Texts with similar meanings should generally have vector representations that are closer together.

For example:

"How do I reset my password?"

and

"I forgot my account password. How can I change it?"

have different words but similar meanings.

An embedding model can represent this semantic similarity mathematically.

Embeddings are extremely important in:

- Semantic search

- Recommendation systems
- RAG systems
- Document retrieval
- Similarity search

9. Vector Databases

A vector database is a specialized database designed to store and search vector embeddings.

Traditional databases are excellent for structured data.

For example:

| User ID | Name | Age |

|---|---|---|

| 1 | Amit | 22 |

| 2 | Rahul | 25 |

However, traditional databases are not primarily designed to search for semantic similarity between high-dimensional vectors.

Vector databases store embeddings and allow similarity searches.

The general process is:

Text Document

↓

Chunking

↓

Embedding Model

↓

Vector Embeddings

↓

Vector Database

When a user asks a question:

User Query

↓

Embedding Model

↓

Query Vector

↓

Similarity Search

↓

Most Relevant Document Chunks

Common similarity measures include:

- Cosine similarity
- Euclidean distance
- Dot product

Vector databases can also store metadata.

For example:

Document:

"Introduction to Machine Learning"

Metadata:

Category: AI

Author: Research Team

Year: 2026

Metadata can help filter search results.

10. Large Language Models

Large Language Models, commonly called LLMs, are deep learning models trained on extremely large amounts of text data.

They are designed to understand and generate language.

Examples of tasks performed by LLMs include:

- Answering questions

- Writing code
- Summarizing documents
- Translating languages
- Generating content
- Explaining concepts
- Analyzing text

The fundamental task of many language models can be described as predicting the next token.

For example:

"The capital of France is..."

The model predicts:

"Paris"

However, modern language models learn complex statistical relationships between words, concepts, syntax, and context.

They do not simply memorize sentences.

During training, models learn patterns from enormous datasets.

11. Transformers

The Transformer architecture is one of the most important developments in modern artificial intelligence.

Transformers became highly influential because they introduced the concept of attention.

Attention allows a model to determine which parts of the input are most relevant to other parts.

Consider the sentence:

"The student studied for the exam because she wanted to pass."

To understand the word "she," the model needs to understand that it refers to "the student."

Attention mechanisms help the model capture these relationships.

The Transformer architecture is commonly associated with:

- Self-attention
- Multi-head attention
- Positional encoding
- Feed-forward neural networks
- Encoder layers
- Decoder layers

Transformer-based architectures are used by many modern language models.

12. Generative AI

Generative AI refers to artificial intelligence systems capable of creating new content.

This content can include:

- Text
- Images
- Audio
- Video
- Code
- Music

A traditional machine learning model might classify an image as "cat."

A generative model can create a completely new image of a cat based on a text description.

Generative AI applications include:

- AI chatbots
- Code assistants
- Image generation
- Voice generation
- Video generation
- Document generation

Generative AI models often require prompts.

A prompt is the input or instruction provided to the model.

For example:

"Explain Retrieval-Augmented Generation in simple language."

The quality and clarity of the prompt can influence the model's output.

13. Prompt Engineering

Prompt engineering is the practice of designing effective instructions for AI models.

A good prompt can include:

- Role
- Context
- Task
- Constraints
- Output format
- Examples

For example:

"You are an AI teacher. Explain vector databases to a beginner. Use simple language and provide one practical example."

This prompt gives the model:

Role: AI teacher

Task: Explain vector databases

Audience: Beginner

Constraint: Use simple language

Additional requirement: Provide an example

Prompt engineering techniques include:

- Zero-shot prompting
- Few-shot prompting
- Chain-of-thought prompting
- Role prompting
- Structured output prompting

14. Limitations of Large Language Models

Although LLMs are powerful, they have important limitations.

One major issue is hallucination.

A hallucination occurs when an AI model generates information that appears believable but is inaccurate or unsupported.

For example, a model might confidently generate:

- A fake source
- An incorrect historical date
- A non-existent product
- An inaccurate technical explanation

Hallucinations can occur because a language model is primarily designed to generate probable text based on patterns.

The model may not automatically verify whether every generated statement is factually correct.

Other limitations include:

- Outdated knowledge
- Limited context windows
- Lack of access to private data
- Difficulty with highly specific information
- Potential bias
- Non-deterministic responses

These limitations are one reason why Retrieval-Augmented Generation was developed.

15. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, commonly known as RAG, is a technique that combines information retrieval with a generative language model.

Instead of asking the LLM to answer entirely from its internal knowledge, a RAG system retrieves relevant information from an external knowledge source.

The retrieved information is then provided to the LLM as additional context.

The process can be represented as:

User Question

↓

Convert Question to Embedding

↓

Search Vector Database

↓

Retrieve Relevant Chunks

↓

Provide Chunks to LLM

↓

Generate Answer

For example, imagine a company has thousands of internal documents.

An employee asks:

"What is the company's remote work policy?"

A normal LLM may not know the company's internal policy.

A RAG system can:

1. Search the company documents.

2. Retrieve the relevant remote work policy.

3. Give the retrieved content to the LLM.

4. Generate an answer based on the provided information.

This makes RAG especially useful for private and domain-specific knowledge.

16. RAG System Architecture

A typical RAG system contains two major phases.

Phase One: Indexing

The indexing phase prepares documents for retrieval.

The process is:

Documents

↓

Document Loader

↓

Text Extraction

↓

Text Chunking

↓

Embedding Model

↓

Vector Database

Documents may include:

- PDF files
- Text files
- Word documents
- Website content
- Database records
- Markdown files

A document loader extracts the content.

The content is then divided into smaller chunks.

Each chunk is converted into an embedding.

The embeddings are stored in a vector database.

Phase Two: Retrieval and Generation

When a user asks a question:

User Question

↓

Embedding Model

↓

Query Embedding

↓

Vector Search

↓

Top Relevant Chunks

↓

Prompt Construction

↓

Large Language Model

↓

Final Answer

The LLM receives both:

1. The user's question

2. The retrieved context

The model then generates an answer using the available information.

17. Document Chunking

Chunking is the process of dividing large documents into smaller pieces.

Chunking is important because:

- LLMs have context limitations.
- Large documents may contain multiple topics.
- Smaller chunks improve retrieval precision.
- Embedding models work better with appropriately sized content.

For example:

A 100-page document should usually not be stored as one massive embedding.

Instead:

Document

↓

Chunk 1

Chunk 2

Chunk 3

Chunk 4

...

Each chunk is embedded separately.

A chunk may contain:

- A paragraph
- Multiple paragraphs
- A fixed number of characters
- A fixed number of tokens

18. Chunk Size and Chunk Overlap

Two important parameters in chunking are:

Chunk Size and Chunk Overlap.

Chunk size determines how much text is included in each chunk.

Chunk overlap determines how much information is shared between consecutive chunks.

For example:

Chunk 1:

"Machine learning allows computers to learn patterns from data."

Chunk 2 with overlap:

"Learn patterns from data. Deep learning uses neural networks with multiple layers."

The overlapping section can help preserve context.

If chunk sizes are too small:

- Important context may be lost.
- The system may retrieve incomplete information.

If chunk sizes are too large:

- Retrieval may become less precise.
- More irrelevant information may be included.

The best chunking strategy depends on:

- Document type
- Content structure
- Embedding model
- User questions
- Retrieval requirements

19. Similarity Search

Similarity search is used to find document chunks that are semantically related to a user's query.

Suppose a vector database contains these chunks:

Chunk A:

"RAG combines document retrieval with language generation."

Chunk B:

"React is a JavaScript library for building user interfaces."

Chunk C:

"Vector databases store numerical embeddings."

A user asks:

"How does RAG find relevant information?"

The embedding of the query will likely be most similar to Chunk A and possibly Chunk C.

The vector database returns the most relevant results.

This is often called Top-K retrieval.

For example:

Top-K = 3

The system retrieves the three most relevant chunks.

Choosing the right value for K is important.

A very small K may miss useful information.

A very large K may provide too much irrelevant context.

20. Retrieval-Augmented Generation and Hallucination Reduction

RAG can reduce hallucinations by providing relevant factual information directly to the language model.

For example:

Without RAG:

User: "What is the vacation policy of Company X?"

The LLM may not know the answer and could generate an incorrect response.

With RAG:

User Question

↓

Search Company Documents

↓

Retrieve Official Vacation Policy

↓

Provide Policy to LLM

↓

Generate Grounded Answer

However, RAG does not completely eliminate hallucinations.

Problems can still occur if:

- The wrong documents are retrieved.
- The relevant document is missing.
- The retrieved context is incomplete.
- The LLM ignores part of the context.
- The knowledge base contains incorrect information.

Therefore, the quality of retrieval is extremely important.

21. Embedding Models in RAG

An embedding model converts text into vector representations.

The same or compatible embedding process is generally used for both:

Document chunks and user queries.

For example:

Document Chunk:

"RAG retrieves relevant documents before generating an answer."

Embedding Model

↓

Document Vector

User Query:

"How does RAG retrieve information?"

Embedding Model

↓

Query Vector

The vector database compares the query vector with stored document vectors.

Semantically similar vectors are retrieved.

The choice of embedding model can significantly affect RAG performance.

A good embedding model should understand semantic similarity in the target domain and language.

22. Metadata Filtering

Metadata provides additional information about document chunks.

Examples include:

- Document name
- Author
- Date
- Category
- Department
- Access level

Suppose a company has documents from multiple departments.

A user from the HR department asks:

"What is the leave policy?"

The RAG system can apply a metadata filter:

Department = HR

This prevents irrelevant documents from other departments from being included in the search.

Metadata filtering can improve:

- Search accuracy
- Performance
- Security
- Organization

23. Hybrid Search

Vector search is excellent for semantic similarity.

However, keyword search can sometimes be better for exact matches.

For example:

A user searching for:

"Error code AUTH_401"

may need an exact keyword match.

Hybrid search combines:

- Keyword search
- Semantic vector search

The results can then be ranked together.

Hybrid search is useful when applications need both semantic understanding and exact terminology matching.

24. Re-Ranking

A retriever may initially return several potentially relevant chunks.

A re-ranking model can then evaluate those chunks more carefully.

The process is:

User Query

↓

Retrieve Top 20 Chunks

↓

Re-Ranker

↓

Select Best 5 Chunks

↓

LLM

Re-ranking can improve retrieval quality.

However, it may increase:

- Latency

- Computational cost
- System complexity

25. Fine-Tuning vs RAG

Fine-tuning and RAG solve different problems.

Fine-tuning modifies the behavior or knowledge patterns of a model through additional training.

RAG retrieves external information at runtime.

Fine-tuning may be useful for:

- Specific writing styles
- Domain-specific behavior
- Specialized tasks
- Consistent output formats

RAG may be useful for:

- Frequently changing information
- Private documents
- Company knowledge
- Large document collections
- Source-based answers

For example, a company's policy documents may change regularly.

Using RAG can be easier than retraining or fine-tuning a model every time a policy changes.

26. AI Agents

An AI agent is a system capable of performing tasks using reasoning, planning, memory, and tools.

A simple chatbot generally follows:

User Input

↓

LLM

↓

Response

An AI agent can perform a more complex process:

User Goal

↓

Analyze Task

↓

Decide What to Do

↓

Use Tools

↓

Observe Results

↓

Continue or Adjust

↓

Final Answer

Tools used by agents may include:

- Web search
- Databases
- APIs
- Calculators
- File systems
- Email systems
- Code execution environments

27. Agentic RAG

Agentic RAG combines RAG systems with AI agents.

Instead of always following a fixed retrieval process, an agent can decide:

- Whether retrieval is necessary
- Which knowledge source to search
- Whether to search again

- Whether retrieved information is sufficient
- Whether to use another tool

For example:

User Question:

"Compare our 2025 and 2026 sales performance and explain the major changes."

An Agentic RAG system may:

1. Identify that multiple documents are required.

2. Search the 2025 sales reports.

3. Search the 2026 sales reports.

4. Retrieve relevant sections.

5. Compare the information.

6. Perform calculations if necessary.

7. Generate a final explanation.

This creates a more flexible system than a simple fixed RAG pipeline.

28. Evaluation of RAG Systems

A RAG system should be evaluated at multiple levels.

Retrieval Evaluation

Important questions include:

- Did the system retrieve the correct document?
- Was the relevant chunk included in the results?
- How relevant were the retrieved chunks?

Generation Evaluation

Important questions include:

- Is the answer factually correct?
- Does the answer use the retrieved context?
- Does it introduce unsupported information?
- Is the answer clear and useful?

Common evaluation concepts include:

- Precision
- Recall
- Faithfulness
- Answer relevance
- Context relevance

Improving only the LLM does not necessarily improve the entire RAG system.

Poor retrieval can cause poor answers even when using a powerful language model.

29. Common Problems in RAG Systems

Several common problems can reduce RAG quality.

Problem 1: Poor Chunking

Large chunks may contain too much unrelated information.

Small chunks may lose context.

Problem 2: Poor Embeddings

An embedding model may fail to understand domain-specific terminology.

Problem 3: Incorrect Retrieval

The vector database may return semantically similar but incorrect documents.

Problem 4: Too Much Context

Providing too many chunks can overwhelm the model with irrelevant information.

Problem 5: Missing Metadata

Without metadata, filtering and document tracking become more difficult.

Problem 6: Hallucination

The LLM may still generate information that is not present in the retrieved documents.

30. Future of RAG and AI Systems

RAG systems are likely to become more sophisticated through:

- Better retrieval algorithms
- Multimodal search
- Agentic workflows
- Real-time knowledge retrieval
- Improved re-ranking
- Knowledge graphs
- Long-context models
- Personalized knowledge systems

Future RAG systems may retrieve not only text but also:

- Images
- Audio
- Videos
- Tables
- Database records
- Real-time information

Multimodal RAG systems will allow users to ask questions about different forms of information.

For example:

A user may upload a financial report containing text, charts, and tables.

A multimodal RAG system could retrieve relevant textual and visual information to answer questions.

31. Conclusion

Artificial Intelligence has evolved from simple rule-based systems into powerful machine learning and deep learning systems capable of understanding and generating human language.

Modern AI systems increasingly rely on Transformer architectures, embeddings, vector databases, and Large Language Models.

Generative AI allows machines to create new content, while Retrieval-Augmented Generation allows language models to access external knowledge.

A typical RAG system performs two major tasks:

First, it processes documents by loading, chunking, embedding, and storing them in a vector database.

Second, it retrieves relevant information when a user asks a question and provides that information to a language model.

RAG is especially useful when an application needs answers based on private, specialized, or frequently changing information.

As AI systems continue to evolve, RAG is increasingly being combined with AI agents, tool calling, memory, re-ranking, hybrid search, and multimodal capabilities.

The result is a new generation of AI applications that can do more than simply generate text. These systems can search knowledge bases, retrieve information, use external tools, reason through complex tasks, and provide responses grounded in relevant data.

Understanding the relationship between machine learning, deep learning, NLP, Transformers, embeddings, vector databases, Large Language Models, Generative AI, RAG, and Agentic AI provides a strong foundation for building modern intelligent applications.